

10시간 집중 과정

파이썬 기초 ~ 응용

빅데이터 분석 도구

데이터 분석을 위한 파이썬 핵심 문법과 라이브러리를 10시간에 압축한 실습 중심 교안

총 10시간 (10모듈) · 실습 환경: Anaconda + VS Code + Jupyter





과정 개요

10시간 커리큘럼 한눈에 보기

- 1 데이터 분석 도구 & 실습환경 구축**
Python·R·Excel 비교 / Anaconda·VS Code
- 2 파이썬 기초 — 변수·자료형·연산**
변수와 상수, 숫자/문자열/불, 연산자
- 3 자료구조 ① — 리스트·튜플**
list / tuple, 인덱싱·슬라이싱
- 4 자료구조 ② — 딕셔너리·세트**
dict / set, 자료구조 비교
- 5 제어문 — 조건문·반복문**
if·elif·else / for·while / break
- 6 함수**
정의·호출, 인자·반환값, lambda
- 7 객체·클래스·모듈/패키지**
OOP, class, import, package
- 8 예외처리·내장/외장함수·라이브러리**
try·except, 내장함수, 표준/외부
- 9 데이터 분석 라이브러리 & 시각화**
NumPy, Pandas, matplotlib
- 10 파일 입출력 & 미니 프로젝트**
표준 입출력, 파일·pickle, 평가



구성 각 모듈 ≈ 1시간. 이론 핵심 설명 후 코드 실습으로 바로 확인하는 '설명 → 실습' 반복 구조입니다.



들 어 가 기 전 에

학습 목표와 수강 대상

이 과정을 마치면

- 파이썬 핵심 문법(자료형·자료구조·제어문·함수)을 자유롭게 사용
- 객체·모듈·라이브러리 개념을 이해하고 코드를 구조화
- NumPy·Pandas·matplotlib로 데이터를 다루고 시각화
- 파일 입출력을 활용한 간단한 응용 프로그램 작성

수강 대상 & 선수 지식

- 데이터 분석에 입문하려는 비전공·타분야 실무자
- 프로그래밍 경험이 없어도 무방 (기초부터 시작)
- **필요한 것 : 노트북, 인터넷, 학습 의지**
- 권장 : 기본적인 컴퓨터·파일 다루기 경험

눈높이 체크 · 지금 얼마나 알고 있나요?



프로그래밍 언어

컴파일러·인터프리터의 차이를 설명할 수 있나요?



표준 라이브러리

파이썬이 기본 제공하는 도구들을 아시나요?



외부 라이브러리

pip로 설치하는 3rd-party 패키지를 써보셨나요?

MODULE 1 / 10



데이터 분석 도구 & 실습환경 구축

왜 파이썬인가, 그리고 어떻게 시작하는가

학습 내용

- 데이터 분석 도구 비교 : Python · R · Excel
- 데이터 분석 언어로서 파이썬의 특징
- 실습환경 구축 : Anaconda · VS Code · Jupyter
- 첫 코드 실행 — print() 와 대화형 인터프리터



1. 데이터 분석 도구

Python · R · Excel 비교

도구	강점	한계
Python	범용 언어, 풍부한 라이브러리. 데이터 핸들링과 AI·머신러닝의 사실상 표준	통계 전용 함수는 R보다 적었으나 최근 대거 보완
R	방대한 통계 패키지, 통계 분석과 시각화에 특화. 학계·통계 전문가 선호	범용성이 낮고 대용량 처리 속도가 느린 편
Excel	누구에게나 친숙, 빠른 집계·그래프. 기업 현장에서 가장 보편적	대용량·고급 분석·반복 자동화에 제약이 큼



결론 범용성·생태계·확장성을 모두 갖춘 Python이 데이터 분석의 중심 언어로 자리 잡았습니다. 이 과정의 주인공입니다.



1. 데이터 분석 도구

데이터 분석 언어로서의 파이썬

파이썬의 특징

- 교육용으로 시작된 대화식 인터프리터 언어
- 간결하고 직관적인 문법 구조
- 작성한 코드를 즉시 실행해 결과 확인
- 방대한 라이브러리·확장 패키지 생태계
- 리스트·딕셔너리 등 풍부한 자료구조 제공
- 다른 언어·데이터베이스와 호환성 우수
- 최근 통계·머신러닝 라이브러리 대거 추가

현재 데이터 분석의 '기본'

- **Python + Pandas = 데이터 분석 표준**
- SQL : DB에서 데이터를 꺼내는 필수 도구
- 시각화는 Power BI · Tableau · matplotlib

TIOBE 프로그래밍 언어 인기 지수

AI·머신러닝·데이터 분석 수요에 힘입어 파이썬은 최근 인기 순위 1위권을 유지하고 있습니다.





1. 실습환경 구성

Anaconda · VS Code · Jupyter



Anaconda 설치

파이썬 + 분석용 패키지(NumPy-Pandas 등)를 한 번에. 가상환경 관리 도구 conda 포함.



VS Code (편집기)

코드 작성·실행·디버깅을 한 곳에서. Python 확장 설치로 자동완성·실행 지원.



Jupyter Notebook

셀 단위로 코드를 실행하고 결과를 즉시 확인. 분석·리포트 작성에 최적.

패키지 설치는 터미널에서 — pip / conda



```
# 외부 라이브러리 설치 (둘 중 하나)
pip install numpy pandas matplotlib
conda install numpy pandas matplotlib
```



1. 첫 코드 실행

print() 와 대화형 인터프리터

```
# 출력 함수
print("Hello, Python!")

# 버전 확인
import sys
print(sys.version)
```

두 가지 실행 방식

- 대화형(인터프리터·Jupyter)
 - 한 줄씩 실행하며 결과 즉시 확인 — 학습에 최적
- 스크립트(.py 파일)
 - 전체를 파일로 저장해 한꺼번에 실행



팁 Jupyter에서는 셀에 마지막 줄 값만 적어도 자동 출력됩니다. 디버깅·탐색에는 대화형, 배포에는 스크립트를 사용하세요.

실습 — 직접 해보기

- Jupyter 노트북을 새로 만들고 print("안녕, 파이썬") 실행
- 본인 이름과 오늘 날짜를 출력해 보기

MODULE 2 / 10



파이썬 기초 변수 · 자료형 · 연산

값을 저장하고 다루는 가장 기본적인 단위

학습 내용

- 변수와 상수 — 값에 이름 붙이기
- 기본 자료형 : 숫자 · 문자열 · 불(bool)
- 연산자 : 산술 · 비교 · 논리
- 자료형 확인과 형 변환



2. 변수와 상수

값에 이름을 붙인다

```
# 변수 = 값을 담는 이름표
name = "홍길동"
age = 25
pi = 3.14

# 여러 변수 동시 할당
x, y, z = 1, 2, 3
```

변수 이름 규칙

- 영문자 또는 _(밑줄)로 시작
- 숫자로 시작 불가, 공백·특수문자 불가
- 예약어(for, if, class 등) 사용 불가
- 대소문자 구분 — Age 와 age 는 다름
- 의미가 드러나는 이름을 권장 (snake_case)



파이썬은 동적 타입 변수 선언 시 자료형을 미리 지정하지 않습니다. 할당된 값에 따라 형이 자동 결정됩니다.

자료형 미지정 — 값으로 형이 결정됨

```
x = 10          # 정수(int)
x = "열"        → 문자열(str)로 바뀜
```



2. 기본 자료형

숫자 · 문자열 · 불(bool)

정수 int

소수점 없는 수

```
1, -7, 100
```

실수 float

소수점 있는 수

```
3.14, -0.5
```

문자열 str

따옴표로 감싼 글자

```
"hi", '파이썬'
```

불 bool

참/거짓 (조건·비교)

```
True, False
```

자료형 확인과 형 변환

```
type(3.14)      # <class 'float'> - 자료형 확인
int("10")       # 문자열 → 정수 10
str(25) + "세"  # 숫자 → 문자열 후 결합 → '25세'
```



2. 연산자

산술 · 비교 · 논리

산술 연산

- + - * / 기본 사칙연산
- // 몫, % 나머지
- ** 거듭제곱
- 예) $7 // 2 = 3$
- $7 \% 2 = 1$
- $2 ** 10 = 1024$

비교 연산

- == 같다, != 다르다
- > < >= <=
- 결과는 True / False
- 예) $3 > 2 \rightarrow \text{True}$
- $5 == 5 \rightarrow \text{True}$
- $1 != 1 \rightarrow \text{False}$

논리 연산

- and 둘 다 참이면 참
- or 하나라도 참이면 참
- not 참/거짓 뒤집기
- 예) True and False
- $\rightarrow \text{False}$
- $\text{not True} \rightarrow \text{False}$



실습 사칙연산 콘솔 프로그램 — 두 수를 입력받아 +, -, *, / 결과를 모두 출력해 보세요. (input() · print() 활용)



자료구조 ① 리스트 · 튜플

여러 값을 하나로 묶어 다루기

학습 내용

- 자료구조란 무엇인가 — 시퀀스형 개요
- 리스트(list) — 변경 가능한 순서 모음
- 튜플(tuple) — 변경 불가능한 순서 모음
- 인덱싱(indexing)과 슬라이싱(slicing)



3. 자료구조

리스트(list) — 변경 가능한 모음



```
# 리스트 생성 — 서로 다른 자료형도 가능
nums = [1, 2, 3]
fruits = ["사과", "배", "감"]

fruits.append("귤")      # 끝에 추가
fruits[0] = "딸기"      # 값 변경 가능
fruits.remove("배")      # 값 삭제
len(fruits)              # 개수 → 3
```

리스트 핵심

- 대괄호 [] 로 생성
- 변경 가능(**mutable**) — 추가·수정·삭제 자유
- 순서가 있어 인덱스로 접근
- 중복 값 허용
- 자주 쓰는 메서드 :
 - append, insert, remove, sort, reverse



활용 데이터 분석에서 가장 많이 쓰는 컨테이너. 여러 값을 순서대로 모아 반복 처리할 때 기본이 됩니다.



3. 자료구조

튜플(tuple) — 변경 불가능한 모음

```
# 튜플 생성 — 소괄호 ( )
point = (3, 5)
rgb = (255, 128, 0)

point[0]          # 읽기는 가능 → 3
point[0] = 9      # 오류! 수정 불가
```

튜플 핵심

- 소괄호 () 로 생성
- 변경 불가(**immutable**) — 생성 후 고정
- 읽기는 리스트와 동일
- 값이 바뀌면 안 되는 데이터에 사용

리스트 vs 튜플 — 언제 무엇을?

리스트 값이 자주 바뀌는 데이터 (장바구니, 분석 대상 목록)

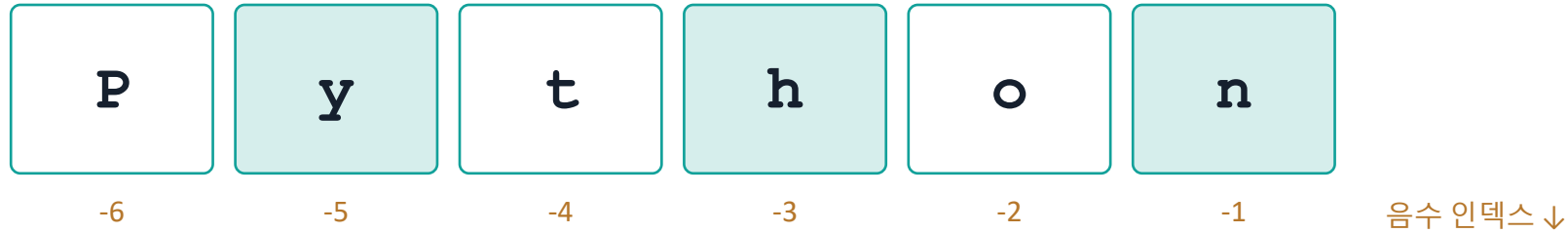
튜플 값이 고정된 데이터 (좌표, RGB 색상, 함수의 여러 반환값)



3. 자료구조

인덱싱 & 슬라이싱

인덱스는 0부터 시작. 음수는 뒤에서부터



```
s = "Python"
s[0]      # 'P'      - 첫 글자
s[-1]     # 'n'      - 마지막 글자
s[0:3]    # 'Pyt'    - 0, 1, 2 (3은 제외)
s[2:]     # 'thon'   - 2부터 끝까지
s[::2]    # 'Pto'    - 2칸씩 건너뛰기
```




자료구조 ② 딕셔너리 · 세트

이름표가 있는 데이터, 중복 없는 데이터

학습 내용

- 딕셔너리(dict) — 키와 값의 쌍
- 세트(set) — 중복을 허용하지 않는 모음
- 각 자료구조의 활용 장면
- 네 가지 자료구조 비교 정리



4. 자료구조

딕셔너리(dict) — 키:값 쌍

```
# 키(key) : 값(value)
person = {
    "name": "홍길동",
    "age": 25
}

person["name"]          # '홍길동'
person["job"] = "개발자" # 추가
person.keys()           # 모든 키
```

딕셔너리 핵심

- 중괄호 {} · key:value 형태
- 키로 값을 즉시 찾음 (순번 아님)
- 키는 중복 불가·고유
- 값은 어떤 자료형도 가능
- JSON·레코드형 데이터의 기본
 - 메서드: keys, values, items, get



활용 한 사람·한 행(row)의 여러 속성을 묶을 때 최적. Pandas의 한 줄도 결국 '키:값'의 모음입니다.



4. 자료구조

세트(set) — 중복 없는 모음



```
# 중복 자동 제거
nums = {1, 2, 2, 3, 3, 3}
print(nums)          # {1, 2, 3}

a = {1, 2, 3}; b = {2, 3, 4}
a & b    # 교집합 {2, 3}
a | b    # 합집합 {1, 2, 3, 4}
```

세트 핵심

- 중괄호 {} (키 없음)
- 중복 불가 — 자동 제거
- 순서 없음 → 인덱싱 불가
- 집합 연산: & | - 지원
- 중복 제거·멤버십 검사에 유용



활용 리스트의 중복을 없앨 때 set(리스트) 한 줄이면 끝. 데이터 정제(cleaning)에서 자주 등장합니다.

예) `unique = list(set([1,1,2,3,3]))` → `[1, 2, 3]`



4. 자료 구조

네 가지 자료구조 비교

자료구조	기호	순서	변경	중복	대표 용도
리스트 list	[]	있음	가능	허용	순서 있는 값 모음, 반복 처리
튜플 tuple	()	있음	불가	허용	고정 데이터(좌표·설정)
딕셔너리 dict	{k:v}	있음*	가능	키 불가	키로 찾는 속성 데이터
세트 set	{}	없음	가능	불가	중복 제거, 집합 연산

* 파이썬 3.7+ 부터 딕셔너리는 입력한 순서를 유지합니다.

 **정리** 데이터의 '성격'에 맞는 그릇을 고르는 것이 좋은 코드의 시작입니다 — 순서·변경·중복·접근 방식을 기준으로 선택하세요.



제어문 조건문 · 반복문

코드의 흐름을 내가 원하는 대로

학습 내용

- 조건문 if · elif · else
- 반복문 for — 정해진 횟수만큼
- 반복문 while — 조건이 참인 동안
- 흐름 제어 : break · continue



5. 제어문

조건문 if · elif · else

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"

print(grade)    # B
```

조건문 규칙

- 조건 뒤에 콜론(:) 필수
- 들여쓰기(보통 공백 4칸)로 블록 구분
- if : 첫 조건
- elif : 추가 조건 (여러 개 가능)
- else : 모두 거짓일 때
- 조건은 True/False로 평가



주의 파이썬은 들여쓰기로 코드 블록을 구분합니다. 들여쓰기가 어긋나면 IndentationError가 발생합니다.



5. 제어문

반복문 for — 정해진 만큼 반복



```
# 리스트 순회
for fruit in ["사과", "배", "감"]:
    print(fruit)
```

```
# range — 0, 1, 2, 3, 4
for i in range(5):
    print(i, i**2) # 제곱표
```

```
# 딕셔너리 순회
for k, v in person.items():
    print(k, v)
```

for 핵심

- for 변수 in 반복가능객체:
- 리스트·튜플·문자열·딕셔너리 모두 순회
- range(n) : 0 ~ n-1 정수 생성
 - range(a, b) : a ~ b-1
 - range(a, b, step) : 간격 지정
- 데이터 분석에서 가장 많이 쓰는 반복



팁 enumerate() 를 쓰면 인덱스와 값을 동시에 얻을 수 있습니다 — for i, v in enumerate(리스트):



5. 제어문

while · break · continue



```
# 조건이 참인 동안 반복
n = 1
total = 0
while n <= 10:
    total += n
    n += 1
print(total)  # 1~10 합 = 55
```

흐름 제어

- **while** 조건:
 - 조건이 거짓이 될 때까지 반복
- **break**
 - 반복을 즉시 완전히 종료
- **continue**
 - 이번 회차만 건너뛰고 다음으로



주의 while은 조건이 계속 참이면 무한 반복됩니다. 반드시 조건을 변화시키는 코드(n += 1 등)를 넣으세요.

함수

반복되는 코드를 묶어 재사용하기

학습 내용

- 함수 정의(def)와 호출
- 매개변수(인자)와 반환값(return)
- 기본값·키워드·가변 인자
- 익명 함수 lambda



6. 함수

정의(def) 와 호출

```
# 함수 정의
def greet(name):
    return "안녕, " + name

# 함수 호출
msg = greet("홍길동")
print(msg)    # 안녕, 홍길동
```

함수를 쓰는 이유

- 재사용 — 한 번 만들고 여러 번 호출
- 중복 제거로 코드가 짧고 명확
- 수정이 한 곳에서 끝남 (유지보수)
- 이름으로 의도가 드러남 (가독성)
- def 이름(매개변수): ... return 값



구조 def 로 정의 → 콜론(:)과 들여쓰기로 본문 → return 으로 결과 반환. return 이 없으면 None을 돌려줍니다.



6. 함수

인자와 반환값

```
# 기본값 인자
def power(x, n=2):
    return x ** n
power(3)          # 9   (n 생략 → 2)
power(2, 10)      # 1024

# 여러 값 반환 (튜플)
def minmax(data):
    return min(data), max(data)
lo, hi = minmax([3, 1, 9, 4])
```

인자 전달 방식

- 위치 인자
 - 순서대로 전달 `power(2, 10)`
- 키워드 인자
 - 이름 지정 `power(x=2, n=10)`
- 기본값 인자
 - 생략 시 기본값 사용 `n=2`
- 여러 값을 반환하면 튜플로 묶여 돌아옵니다



6. 함수

가변 인자 · 익명 함수 lambda



```
# 가변 인자 *args - 개수 제한 없음
def total(*nums):
    return sum(nums)
total(1, 2, 3, 4)    # 10

# lambda - 한 줄짜리 익명 함수
square = lambda x: x * x
square(5)            # 25
```

언제 lambda?

- 이름 없이 즉석에서 쓰는 짧은 함수
- **sorted, map, filter** 의 **key**로 자주 사용
 - `sorted(data, key=lambda x: x[1])`
- 본문이 길어지면 def로 정의하는 게 명확



실습 리스트 `[("A",80),("B",95),("C",70)]` 를 점수 기준 내림차순으로 정렬해 보세요. (lambda 활용)

MODULE 7 / 10



객체 · 클래스 모듈 · 패키지

코드를 더 크게, 더 체계적으로

학습 내용

- 객체지향 프로그래밍(OOP) 개념
- 클래스 정의와 객체 생성
- 모듈(module)과 import
- 패키지(package)와 라이브러리 구조



클래스로 데이터와 기능을 묶기

```
class Dog:
    def __init__(self, name):
        self.name = name # 속성

    def bark(self): # 메서드
        return self.name + "멍멍!"

# 객체 (인스턴스) 생성
d = Dog("바둑이")
d.bark() # 바둑이멍멍!
```

객체지향 핵심 개념

- 클래스 = 설계도
 - 객체 = 설계도로 만든 실제 물건
- 속성(attribute)
 - 객체가 가진 데이터 (self.name)
- 메서드(method)
 - 객체가 하는 동작 (bark)
- `__init__`: 객체 생성 시 자동 호출



7. 모듈과 패키지

코드를 파일로 나누고 가져오기

```
# 모듈 전체 가져오기
import math
math.sqrt(16)           # 4.0

# 별칭(as) - 관례적인 줄임 이름
import numpy as np

# 일부만 가져오기
from math import pi, sqrt
```

용어 정리

- **모듈(module)**
 - 함수·변수를 모아둔 하나의 .py 파일
- **패키지(package)**
 - 여러 모듈을 묶은 폴더 구조
- **라이브러리(library)**
 - 재사용 가능한 모듈·패키지의 모음



흐름 함수 → 모듈(파일) → 패키지(폴더) → 라이브러리. 코드를 점점 더 큰 단위로 묶어 재사용하는 구조입니다.

MODULE 8 / 10



예외처리 · 함수 라이브러리

오류에 강한 코드, 이미 만들어진 도구

학습 내용

- 예외처리 try · except · finally
- 자주 쓰는 내장 함수(built-in)
- 문자열 처리 함수
- 표준 라이브러리와 외부 라이브러리



8. 예외처리

오류가 나도 멈추지 않게

```
try:
    n = int(input("숫자: "))
    print(10 / n)
except ZeroDivisionError:
    print("0으로 나눌 수 없어요")
except ValueError:
    print("숫자가 아니에요")
finally:
    print("끝!")
```

구문 역할

- **try**
 - 오류가 날 수 있는 코드
- **except**
 - 오류 종류별 대응 (여러 개 가능)
- **finally**
 - 성공·실패와 무관하게 항상 실행



왜 필요한가 사용자 입력·파일·네트워크처럼 예측 불가능한 상황을 안전하게 처리해 프로그램이 갑자기 죽지 않게 합니다.



내장 함수 · 문자열 처리

자주 쓰는 내장 함수

- len() 길이·개수
- print() / input() 출력·입력
- type() 자료형 확인
- int() float() str() 형 변환
- range() 연속 정수 생성
- sum() min() max() 합·최소·최대
- sorted() 정렬, enumerate() 번호
- abs() round() 절댓값·반올림

문자열 처리 함수(메서드)

```
s = " Hello Python "
```

s.strip()	# 양끝 공백 제거
s.upper()	# 대문자
s.lower()	# 소문자
s.replace("o","0")	
s.split()	# 공백 기준 분리 → 리스트
",".join(["a","b"])	# 'a,b'
s.find("Py")	# 위치 인덱스



정리 내장 함수는 설치 없이 바로 쓸 수 있는 파이썬의 기본 도구입니다. 데이터 전처리의 8할이 문자열 처리에서 시작됩니다.



8. 라이브러리

표준 라이브러리 vs 외부 라이브러리

표준 라이브러리 (내장)

- 설치 없이 import 만으로 사용
- math 수학 함수
- random 난수·무작위 추출
- datetime 날짜·시간
- os / sys 운영체제·시스템
- json JSON 데이터 처리

외부 라이브러리 (3rd-party)

- pip / conda 로 별도 설치 필요
- NumPy 수치 계산·배열
- Pandas 표 형태 데이터 분석
- matplotlib 시각화
- scikit-learn 머신러닝
- requests / Flask 웹



```
import random          # 표준 - 설치 불필요
lotto = random.sample(range(1, 46), 6) # 로또 6개 무작위 추출
```

MODULE 9 / 10



데이터 분석 라이브러리 & 시각화

실전 데이터 분석의 3종 세트

학습 내용

- NumPy — 빠른 수치 계산과 배열
- Pandas — 표(DataFrame) 데이터 분석
- matplotlib — 데이터 시각화
- 막대·선·원 그래프 그리기



9. NUMPY

빠른 수치 계산과 배열

```
import numpy as np

a = np.array([1, 2, 3, 4])
a * 2          # [2 4 6 8]   전체 연산
a.mean()       # 2.5   평균
a.sum()        # 10   합계

# 무작위 추출
np.random.choice(range(1, 46), 6)
```

NumPy 핵심

- ndarray — 고성능 다차원 배열
- 반복문 없이 배열 전체를 한 번에 계산
- C로 구현돼 순수 파이썬보다 훨씬 빠름
- 평균·합·표준편차 등 통계 함수 내장
- Pandas·머신러닝의 토대가 되는 라이브러리



관례 보통 `import numpy as np` 로 'np' 별칭을 붙여 사용합니다. 거의 모든 예제 코드가 이 관례를 따릅니다.



표(DataFrame) 데이터 분석

```
import pandas as pd

df = pd.DataFrame({
    "이름": ["A", "B", "C"],
    "점수": [90, 85, 70]
})

df.head()           # 앞부분 미리보기
df["점수"].mean()   # 평균 점수
df[df["점수"] >= 80] # 조건 필터링
```

Pandas 핵심

- DataFrame — 엑셀 표 같은 2차원 구조
- R의 data.frame을 본떠 설계
- 행·열 선택, 필터, 정렬, 그룹 집계
- CSV·엑셀·DB를 손쉽게 읽고 쓰기
 - read_csv() 한 줄로 데이터 불러오기
- 데이터 분석 실무의 중심 도구

➔ 흐름 데이터 불러오기 → 정제 → 집계·분석 → 시각화. 이 과정의 거의 모든 단계를 Pandas가 담당합니다.



9. MATPLOTLIB

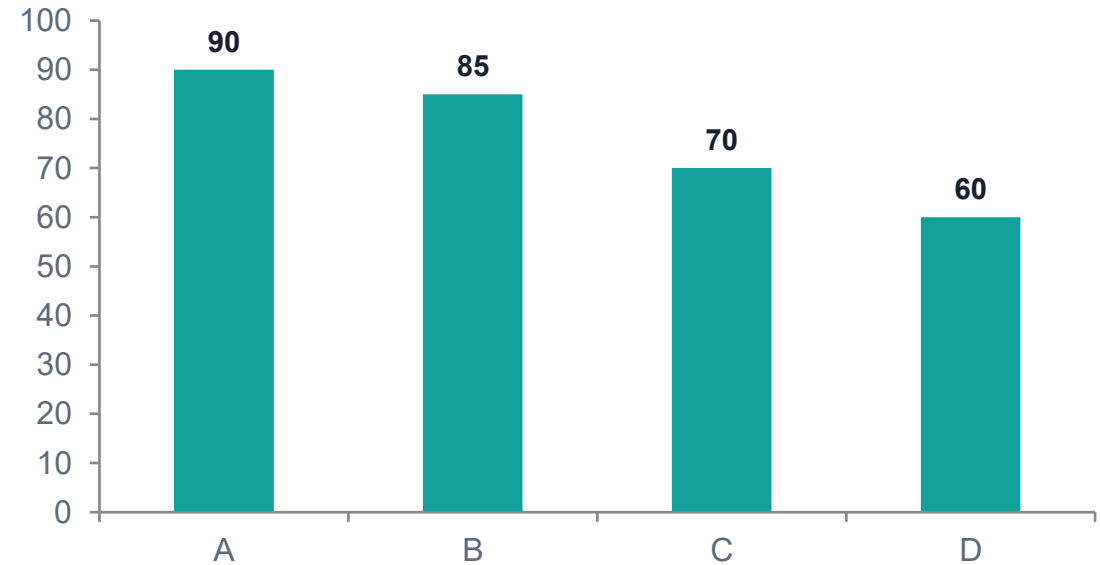
데이터 시각화

```
import matplotlib.pyplot as plt

x = ["A", "B", "C", "D"]
y = [90, 85, 70, 60]

plt.bar(x, y)          # 막대 그래프
plt.title("점수")
plt.show()
```

결과 — 막대 그래프



matplotlib 외에 seaborn·plotly 등으로 더 다양하고 인터랙티브한 시각화도 가능합니다.



지원 그래프 선(plot)·막대(bar)·히스토그램(hist)·산점도(scatter)·원(pie) 등 — 분석 목적에 맞는 그래프를 선택하세요.

MODULE 10 / 10



파일 입출력 & 미니 프로젝트

배운 것을 모아 작은 프로그램으로

학습 내용

- 표준 입출력과 문자열 형식화(format)
- 파일 읽기·쓰기와 pickle
- 미니 프로젝트 — 친구 관리 프로그램
- 다음 단계 : GUI · 웹(Flask) 맛보기



10. 표준 입출력

입력 · 출력 · 문자열 형식화

```
# 입력 — 항상 문자열로 들어옴
name = input("이름: ")
age = int(input("나이: "))

# f-string — 가장 깔끔한 형식화
print(f"{name}님은 {age}세")

# 소수 자리수 지정
print(f"평균: {3.14159:.2f}") # 3.14
```

형식화 방법

- **f-string (권장)**
 - `f"{변수}"` — 가장 직관적
- **.format()**
 - `"{}".format(값)`
- **% 연산자 (옛 방식)**
 - `"%d" % 값`



주의 input()의 결과는 항상 문자열입니다. 숫자로 계산하려면 int()·float()로 변환해야 합니다.



10. 파일 입출력

파일 읽기 · 쓰기 · pickle

```
# 쓰기 - with 블록이 자동으로 닫아줌
with open("memo.txt", "w", encoding="utf-8") as f:
    f.write("안녕\n")

# 읽기
with open("memo.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

핵심 정리

- **open(파일, 모드)**
 - "r" 읽기 "w" 쓰기(덮어씀) "a" 추가
- **with 문 사용 권장**
 - 블록을 벗어나면 파일 자동 닫힘
- **한글은 encoding="utf-8"**
- **pickle**
 - 리스트·딕셔너리 등 객체를 그대로 저장



활용 분석 결과·설정·중간 데이터를 파일로 남기면 프로그램을 다시 실행해도 이어서 작업할 수 있습니다.



10. 미니 프로젝트

친구 관리 프로그램 만들기

지금까지 배운 자료구조·반복문·함수·파일 입출력을 모두 사용합니다.

```
friends = [] # 리스트

def add(name): # 추가
    friends.append(name)

def remove(name): # 삭제
    if name in friends:
        friends.remove(name)

def show(): # 출력
    for i, n in enumerate(friends, 1):
        print(f"{i}. {n}")
```

구현 기능 (요구사항)

- 1. 친구 리스트 출력
- 2. 친구 추가
- 3. 친구 삭제
- 4. 이름 변경
- 5. 파일로 저장·불러오기
- 6. 종료
- → while 루프 + 메뉴 선택(if)으로
- 하나의 콘솔 프로그램 완성



10. 다음 단계

더 나아가기 — GUI · 웹



GUI 데스크톱 앱

Tkinter · PyQt 로 버튼·입력창이 있는 창 프로그램 제작



웹 애플리케이션

Flask 가상환경으로 간단한 웹 서버·API 구축



데이터 과학 · AI

scikit-learn · TensorFlow 로 머신러닝·딥러닝 입문



방향 이 과정은 '기초 체력'을 만드는 단계입니다. 관심 분야(웹·데이터·AI)를 정해 하나씩 깊이 파고드세요.

학습을 이어가는 가장 좋은 방법은 '작게라도 직접 만들어 보는 것'입니다.



마무리

정리 · 자기 점검 · 평가

10시간 동안 다룬 것

- 기초 : 변수·자료형·연산
- 자료구조 : 리스트·튜플·딕셔너리·세트
- 흐름 : 조건문·반복문·함수
- 구조화 : 객체·클래스·모듈·라이브러리
- 실전 : NumPy·Pandas·matplotlib
- 응용 : 파일 입출력·미니 프로젝트

자기 점검 (Self-Check)

- 리스트와 딕셔너리의 차이를 설명할 수 있다
- for / while 을 상황에 맞게 쓸 수 있다
- 함수를 정의하고 인자·반환값을 다룰 수 있다
- 외부 라이브러리를 설치·import 할 수 있다
- CSV를 Pandas로 읽고 간단히 분석할 수 있다



필기 평가

자료형·자료구조·제어문·함수 개념 확인 문항



실기 평가

미니 프로젝트(친구 관리 프로그램) 직접 구현·실행

수고하셨습니다

이제 파이썬으로 데이터를 다루는 첫 걸음을 떼셨습니다.

작은 코드라도 매일 직접 써보는 것이 가장 빠른 성장의 길입니다.

파이썬 기초 ~ 응용 · 빅데이터 분석 도구 (10시간 과정)

